

Projektdokumentation

Entwicklung einer Software zur Lagerverwaltung von medizinischem Verbrauchsmaterial in einem Krankenhaus

JavaFX · MySQL · MVC- und DAO-Architektur

Verfasser	Sascha Schulz
Gruppe	2551
Ausbildungsberuf	Wirtschaftsinformatiker (WI)
Ausbildungsstätte	SRH Berufliche Rehabilitation GmbH
Betreuerin / Ausbilderin	Frau Cramer
Projektzeitraum	04.05.2026 – 26.06.2026 (6 Wochen, 180 h geplant)
Technologien	Java 17, JavaFX 21, MySQL 8, JDBC, MVC-Pattern
Abgabedatum	17.07.2026 (Moodle-Upload als PDF)

Inhaltsverzeichnis

1 Einleitung	5
1.1 Projektbeschreibung	5
1.2 Projektziel	5
1.3 Projektumfeld	5
1.4 Projektbegründung	6
2 Projektplanung	6
2.1 Projektphasen	6
2.2 Ressourcenplanung	6
2.3 Entwicklungsprozess	7
3 Analysephase	7
3.1 Ist-Analyse	7
3.2 Anwendungsfälle	7
3.3 Lastenheft	8
4 Entwurfsphase	8
4.1 Datenbankentwurf	8
4.2 Architekturentwurf (MVC + DAO)	9
4.3 Entwurf der Benutzeroberfläche	10
4.4 Pflichtenheft	11
5 Implementierungsphase	11
5.1 Datenbank und JDBC-Verbindung	11
5.2 DAO-Schicht und Model-Klassen	12
5.3 Benutzeroberfläche und CRUD	12
5.4 Such- und Filterfunktion	12
5.5 Mindestbestand-Warnung und Lagerbuchungen	13
6 Abnahme- und Testphase	13
6.1 Testfälle und Testergebnisse	13
6.2 Soll-Ist-Vergleich	14
6.3 Abnahme und Übergabe	14
7 Dokumentation	15
7.1 Benutzerhandbuch	15
7.2 Entwicklerdokumentation	15
8 Fazit	15
8.1 Soll-Ist-Vergleich der Projektziele	15
8.2 Lessons Learned	15
8.3 Ausblick	16
Abgabeerklärung	17
Quellenverzeichnis	18
Anhang	19
Anhang A – ER-Diagramm	19
Anhang B – SQL-Datenbankskript	19

Anhang C – Benutzerhandbuch.....	21
----------------------------------	----

Abbildungsverzeichnis

Abbildung 1: Use-Case-Diagramm der Hauptfunktionen.....	8
Abbildung 2: ER-Diagramm der Datenbank (6 Tabellen, 3NF).....	9
Abbildung 3: Schichtenarchitektur (MVC mit DAO-Schicht).....	10
Abbildung 4: Entwurf des Hauptfensters (Bestandsübersicht).....	10
Abbildung 5: ER-Diagramm der Datenbank (Anhang A).....	19

Tabellenverzeichnis

Tabelle 1: Beteiligte Rollen und Auftragsschnittstellen.....	5
Tabelle 2: Projektphasen und Stundenverteilung.....	6
Tabelle 3: Risiken und Gegenmaßnahmen.....	6
Tabelle 4: Pflichtanforderungen laut Ausbildungsrichtlinie.....	8
Tabelle 5: Krankenhausspezifische Zusatzanforderungen.....	8
Tabelle 6: Datenbanktabellen im Überblick.....	9
Tabelle 7: Zuordnung der Klassen zu den MVC-Schichten.....	12
Tabelle 8: Definierte Testfälle (Auszug).....	13
Tabelle 9: Soll-Ist-Vergleich der Anforderungen.....	14

Quellcodeverzeichnis

Listing 1: CREATE-Statement der Tabelle materialien.....	11
Listing 2: Singleton-Datenbankverbindung (DBConnection).....	11
Listing 3: CRUD-Beispiel: create() in MaterialDAO.....	12
Listing 4: Filterung der Bestandsübersicht nach Name und Station.....	13
Listing 5: Mindestbestand-Warnung über eine RowFactory.....	13
Listing 6: Datenbankschema – CREATE-Statements aller sechs Tabellen.....	20

1 Einleitung

1.1 Projektbeschreibung

Im Rahmen der Do-IT-Projektphase der SRH Beruflichen Rehabilitation GmbH entstand eine Desktop-Anwendung zur Lagerverwaltung von medizinischem Verbrauchsmaterial in einem Krankenhaus. Verwaltet werden zum Beispiel Verbandsmaterial, Spritzen und Einmalhandschuhe. Die Anwendung wurde mit Java 17 und JavaFX als Oberfläche sowie einer MySQL-Datenbank umgesetzt und läuft als Einzelplatzlösung ohne Internetanbindung.

Die Software gibt dem Personal jederzeit einen klaren Überblick über die Bestände der einzelnen Stationen, warnt bei knappen Materialien und protokolliert jede Ein- und Ausbuchung lückenlos.

1.2 Projektziel

Ziel war eine einfach bedienbare Anwendung, mit der das Krankenhauspersonal Materialien anlegen, ändern und löschen (CRUD), Bestände einsehen, nach Materialname suchen, nach Station filtern sowie bei Unterschreitung des Mindestbestands gewarnt werden kann. Fachlich gefordert waren eine MySQL-Datenbank mit mindestens fünf Tabellen, vollständige CRUD-Funktionalität, eine MVC-Architektur und eine Such- bzw. Filterfunktion. Diese Pflichtanforderungen bilden den Kern des Projektziels.

1.3 Projektumfeld

Das Projekt wurde als Einzelprojekt im Rahmen der WI-Ausbildung (Gruppe 2551) durchgeführt. Die fachliche Begleitung, Bewertung und Abnahme erfolgt durch die Betreuerin Frau Cramer. Inhaltlich orientiert sich das Vorhaben an einem realistischen Anwendungsfall aus dem Krankenhaus; Auftraggeber und Endnutzer (Krankenhausleitung, Pflege- und Stationspersonal) sind fiktiv und dienen der praxisnahen Darstellung.

Tabelle 1: Beteiligte Rollen und Auftragsschnittstellen

Rolle	Person	Funktion
Betreuerin / Ausbilderin	Frau Cramer	Fachliche Begleitung, Bewertung, Abnahme
Fiktiver Auftraggeber	Krankenhausleitung	Anforderungssteller für die Software
Fiktive Endnutzer	Pflege- und Stationspersonal	Tägliche Nutzung der Materialverwaltung

1.4 Projektbegründung

In vielen Häusern wird Verbrauchsmaterial bis heute von Hand in Listen oder Tabellen gepflegt. Das ist fehleranfällig und unübersichtlich: Es fehlt der schnelle Blick auf den Bestand je Station, ein automatischer Hinweis bei knappen Materialien und eine nachvollziehbare Buchungshistorie. Eine kleine, fokussierte Software schafft hier Abhilfe. Der Nutzen liegt in weniger Fehlbeständen, schnellerem Auffinden von Material und einer prüfbaren Historie für die Verwaltung.

2 Projektplanung

2.1 Projektphasen

Das Projekt umfasst sieben Phasen mit einem geplanten Gesamtaufwand von 180 Stunden. Die Anwesenheit beträgt sechs Wochen mit je 32 Stunden (192 h); die Differenz von 12 Stunden dient als Puffer. Die Aufteilung folgt dem Projektantrag (v2) und dem Leitfaden der SRH.

Tabelle 2: Projektphasen und Stundenverteilung

#	Phase	Inhalt	h
1	Analyse & Planung	Anforderungen, ER-Diagramm, DB-Schema, Wochenplan	20
2	Datenbank	SQL-Skripte, 6 Tabellen, Testdaten, JDBC-Verbindung	20
3	DAO & Model	MVC-Architektur, alle DAO- und Model-Klassen	20
4	UI-Grundfunktionen	Hauptfenster, CRUD für alle 6 Entitäten, Tabellen, Dialoge	40
5	UI erweitert	Suche nach Name, Filter nach Station, Mindestbestand-Warnung	25
6	Test & Bugfixes	Funktionen testen, Fehler beheben, Code-Review	25
7	Dokumentation	Benutzerhandbuch, Projektdoku, Abgabe (Moodle)	30
Σ	Gesamt		180

2.2 Ressourcenplanung

Das Projekt wurde von einer Person verantwortet, die Analyse, Entwicklung, Test und Dokumentation übernahm. Als Sachmittel kamen ein privater Laptop (Windows 10/11, ≥ 8 GB RAM), Java 17, JavaFX 21, MySQL 8 mit Connector/J, IntelliJ IDEA, Maven und Git zum Einsatz; alle Werkzeuge sind kostenfrei. Zur Übung der Kalkulation wurde eine fiktive Kostenrechnung erstellt: 150 h Entwicklung und Test zu einem fiktiven Satz von 80,00 €/h zuzüglich anteiliger Sachkosten ergeben rund 12.150,00 €. Tatsächliche Kosten entstehen in der Ausbildung nicht.

Tabelle 3: Risiken und Gegenmaßnahmen

Risiko	Wahrsch.	Gegenmaßnahme
JDBC-Verbindungsprobleme	Mittel	Früher Test, Singleton-DBConnection

Risiko	Wahrsch.	Gegenmaßnahme
JavaFX-Komplexität (TableView)	Hoch	Einfache Layouts zuerst, schrittweise erweitern
Zeitverzug in Phase 4	Mittel	12 h Puffer, Kernanforderungen zuerst
Datenverlust im Test	Niedrig	Testdatenbank trennen, Backups

2.3 Entwicklungsprozess

Entwickelt wurde schrittweise entlang der Phasen. Zuerst entstanden ein tragfähiges Datenmodell und die Datenzugriffsschicht, danach die Oberfläche. Innerhalb der Oberfläche wurden die Kernmodule (Materialien, Kategorien, Stationslager) vollständig umgesetzt, bevor die optionalen Module (Lieferanten, Bestellungen, Bestandsbewegungen) folgten. So war früh ein lauffähiger Kern vorhanden, der gegen die Pflichtenforderungen geprüft werden konnte. Jede wesentliche Entscheidung und Abweichung wurde in einem Projekttagbuch festgehalten.

3 Analysephase

3.1 Ist-Analyse

Ausgangspunkt ist die manuelle Materialverwaltung per Liste oder Tabellenkalkulation. Daraus ergeben sich mehrere Probleme: kein schneller Überblick über die Bestände der einzelnen Stationen (Station 1, OP, ICU, Zentrallager), keine automatische Warnung bei Unterschreitung des Mindestbestands, keine lückenlose Buchungshistorie für die Nachverfolgung, manuelle Fehler bei der Bestandspflege und keine Filterung nach Station oder Materialart bei der Suche.

3.2 Anwendungsfälle

Aus der Ist-Analyse wurden die zentralen Anwendungsfälle abgeleitet. Das Personal verwaltet Materialien (anlegen, bearbeiten, löschen), sieht in der Bestandsübersicht den je Material und Lager berechneten Bestand ein, sucht nach Name und filtert nach Station und erkennt Warnungen bei knappem Bestand. Außerdem werden Bestellungen erfasst sowie Ein- und Ausgänge gebucht: Entnahmen über die Funktion Warenentnahme, Umlagerungen zwischen Lagern als Ausgang und Eingang.

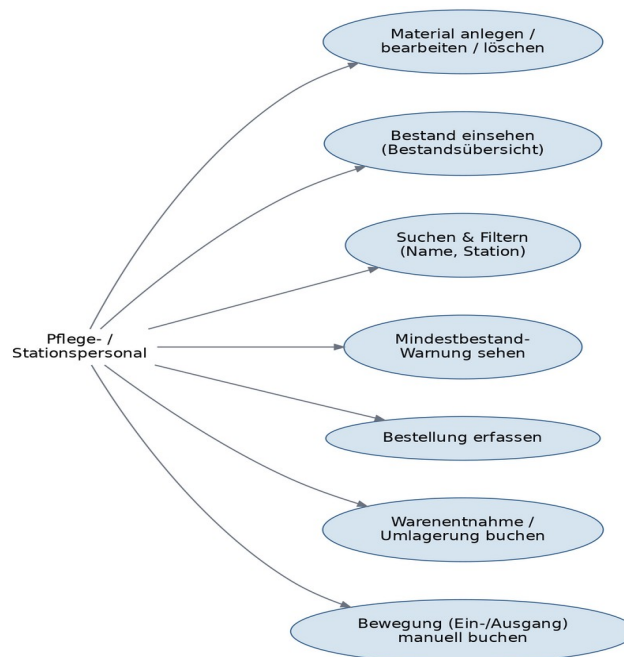


Abbildung 1: Use-Case-Diagramm der Hauptfunktionen

3.3 Lastenheft

Das Lastenheft fasst die Anforderungen zusammen. Die Pflichtanforderungen ergeben sich aus der Ausbildungsrichtlinie, die Zusatzanforderungen aus dem Krankenhauskontext.

Tabelle 4: Pflichtanforderungen laut Ausbildungsrichtlinie

Pflichtanforderung	Beschreibung
Datenbankanbindung	MySQL-Datenbank mit mindestens 5 Tabellen
CRUD-Operationen	Erstellen, Lesen, Ändern, Löschen für alle Entitäten
MVC-Architektur	Klare Trennung von Model, View und Controller
Such- und Filterfunktion	Suche nach Materialname, Filter nach Station

Tabelle 5: Krankenhausspezifische Zusatzanforderungen

Zusatzanforderung	Beschreibung
Mindestbestand-Warnung	Visuelle Hervorhebung bei Unterschreitung
Buchungshistorie	Protokoll aller Ein- und Ausgänge
Stationsfilter	Anzeige nach Krankenhaus-Station
Benutzerfreundlichkeit	Intuitive Oberfläche für nicht-technisches Personal

4 Entwurfsphase

4.1 Datenbankentwurf

Auf Basis der Anforderungen wurde das Datenmodell in sechs normalisierten Tabellen (3. Normalform) entworfen. Der Bestand wird bewusst nicht im Material gespeichert, sondern aus

den Bewegungen berechnet (Summe EINGANG minus Summe AUSGANG). So bleibt das Modell konsistent und die Historie lückenlos und prüfbar.

Tabelle 6: Datenbanktabellen im Überblick

Tabelle	Zweck
kategorien	Materialgruppen (z. B. Verband, Injektion)
lieferanten	Medizinische Lieferanten mit Kontaktdaten
stationslager	Stationen und Lagerorte im Krankenhaus
materialien	Stammdaten aller Materialien inkl. Mindestbestand
bestellungen	Bestellhistorie mit Status und Lieferdatum
bestandsbewegungen	Buchungshistorie (EINGANG/AUSGANG); Bestand daraus berechnet

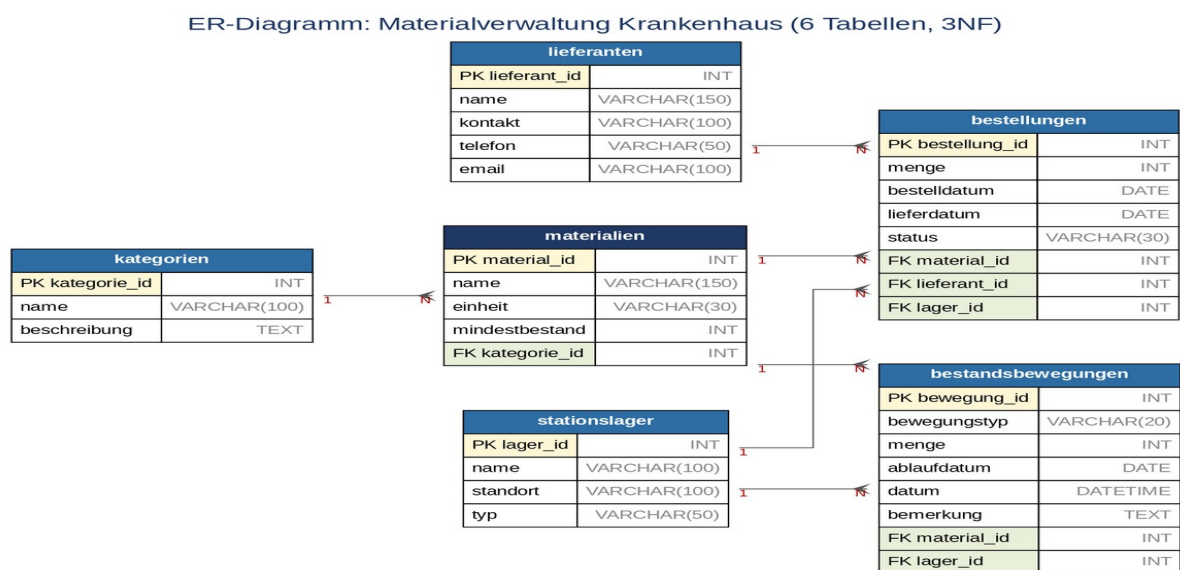


Abbildung 2: ER-Diagramm der Datenbank (6 Tabellen, 3NF)

4.2 Architekturentwurf (MVC + DAO)

Die Anwendung folgt dem Muster Model-View-Controller mit einer zusätzlichen DAO-Schicht für den Datenbankzugriff. Die View (JavaFX/FXML) zeigt Daten und nimmt Eingaben entgegen, der Controller steuert die Bedienlogik, die DAO-Klassen kapseln alle SQL-Zugriffe, und die Model-Klassen sind einfache Datenobjekte (POJOs). Diese Trennung macht den Code übersichtlich und gut testbar.

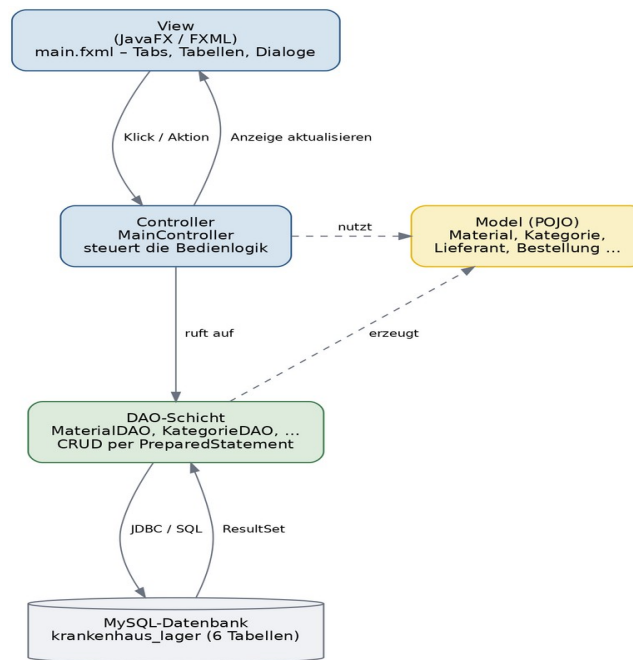


Abbildung 3: Schichtenarchitektur (MVC mit DAO-Schicht)

4.3 Entwurf der Benutzeroberfläche

Die Oberfläche wurde als ein Hauptfenster mit sieben Registerkarten (TabPane) entworfen. Die erste Registerkarte „Bestandsübersicht“ enthält oben eine Such- und Filterleiste (Suchfeld für den Namen, Auswahl der Station) und darunter die Bestandstabelle; Zeilen mit zu niedrigem Bestand werden rot hervorgehoben. Die weiteren Registerkarten (Materialien, Kategorien, Stationslager, Lieferanten, Bestellungen, Bestandsbewegungen) bieten je Entität die Schaltflächen Anlegen, Bearbeiten und Löschen.

Materialverwaltung Krankenhaus - Bestandsuebersicht

Bestandsuebersicht

Materialien

Kategorien

Stationslager

Lieferanten

Bestellungen

Bestandsbewegungen

Suche: Materialname ...

Station v

Suchen

Zuruecksetzen

Material	Einheit	Bestand	Mindest	Status
Einmalhandschuhe M	Stk	420	100	OK
Mullkompressen	Pkg	18	50	WARNUNG
Einwegspritzen 5ml	Stk	250	80	OK
Desinfektionsmittel	Fl	12	30	WARNUNG

Aktualisieren

Warenentnahme

Umlagerung

Rot markierte Zeile = Bestand <= Mindestbestand (Status WARNUNG)

Abbildung 4: Entwurf des Hauptfensters (Bestandsübersicht)

4.4 Pflichtenheft

Das Pflichtenheft beschreibt, wie die Anforderungen technisch umgesetzt werden: die sechs Tabellen als MySQL-Schema, die CRUD-Operationen je Entität über PreparedStatements, die MVC-Struktur mit DAO-Schicht, die Filterung der Bestandssicht nach Name und Station sowie die Mindestbestand-Warnung über eine RowFactory mit Zeilenfärbung. Die optionalen Module (Lieferanten, Bestellungen, Bestandsbewegungen) werden nur bei vollständigem Kern umgesetzt.

5 Implementierungsphase

Dieses Kapitel zeigt die Umsetzung anhand repräsentativer Code-Auszüge. Der vollständige Quellcode liegt dem Abgabepaket bei.

5.1 Datenbank und JDBC-Verbindung

Das CREATE-Skript legt alle sechs Tabellen mit Primär- und Fremdschlüsseln an. Beispielhaft die Haupttabelle materialien:

```
CREATE TABLE materialien (  
  material_id INT AUTO INCREMENT PRIMARY KEY,  
  name VARCHAR(150) NOT NULL,  
  einheit VARCHAR(30) NOT NULL,  
  mindestbestand INT NOT NULL DEFAULT 0,  
  kategorie_id INT NOT NULL,  
  FOREIGN KEY (kategorie_id)  
    REFERENCES kategorien(kategorie_id)  
);
```

Listing 1: CREATE-Statement der Tabelle materialien

Die Datenbankverbindung ist als Singleton umgesetzt, damit nur eine wiederverwendbare Verbindung besteht:

```
public class DBConnection {  
  private static final String URL =  
    "jdbc:mysql://127.0.0.1:3324/krankenhaus_lager"  
    + "?serverTimezone=Europe/Berlin";  
  private static Connection connection;  
  public static Connection getConnection() throws SQLException {  
    if (connection == null || connection.isClosed())  
      connection = DriverManager.getConnection(URL, "root", "1234");  
    return connection;  
  }  
}
```

Listing 2: Singleton-Datenbankverbindung (DBConnection)

5.2 DAO-Schicht und Model-Klassen

Jede DAO-Klasse setzt die CRUD-Operationen um. Lesende Zugriffe liefern Listen über `findAll()`; schreibende Zugriffe nutzen PreparedStatements gegen SQL-Injection. Beispiel für das Anlegen eines Materials:

```
public Material create(Material material) throws SQLException {
    String sql = "INSERT INTO materialien (name, einheit, "
        + "mindestbestand, kategorie_id) VALUES (?, ?, ?, ?)";
    Connection conn = DBConnection.getConnection();
    try (PreparedStatement ps = conn.prepareStatement(sql,
        Statement.RETURN_GENERATED_KEYS)) {
        ps.setString(1, material.getName());
        ps.setString(2, material.getEinheit());
        ps.setInt(3, material.getMindestbestand());
        ps.setInt(4, material.getKategorieId());
        ps.executeUpdate();
    }
    return material;
}
```

Listing 3: CRUD-Beispiel: create() in MaterialDAO

Tabelle 7: Zuordnung der Klassen zu den MVC-Schichten

Schicht	Inhalt
Model	Material, Kategorie, Lieferant, Stationslager, Bestellung, Bestandsbewegung
View	JavaFX-FXML (main.fxml): Hauptfenster mit Registerkarten je Entität
Controller	MainController: steuert die UI-Logik, delegiert an die DAO-Schicht
DAO	MaterialDAO, KategorieDAO, LieferantDAO, StationslagerDAO, ...

5.3 Benutzeroberfläche und CRUD

Das Hauptfenster wurde mit einem TabPane realisiert. Die Bestandsübersicht zeigt je Material und Lager den aus den Bewegungen berechneten Bestand (per JOIN über `findBestandViews`) mit den Spalten Material, Einheit, Bestand, Mindestbestand und Status. Die Registerkarte Materialien verwaltet die Stammdaten (Name, Einheit, Mindestbestand, Kategorie-ID). Über Anlegen, Bearbeiten und Löschen werden Datensätze in einem Dialog gepflegt; das vollständige CRUD wurde für alle sechs Entitäten umgesetzt.

5.4 Such- und Filterfunktion

Die Anzeige der Bestandsübersicht wird gefiltert: Das Suchfeld (Materialname) und die Stationsauswahl wirken zusammen; angezeigt wird nur, was beide Bedingungen erfüllt. Ausgelöst wird die Filterung über die Eingabetaste oder die Schaltfläche „Suchen“:

```
private void anzeigeFiltern() {
    String suchtext = txtSuche.getText() == null
        ? "" : txtSuche.getText().trim().toLowerCase();
    Stationslager lager = cmbLager.getValue();
    ObservableList<BestandView> gefiltert =
```

```

        FXCollections.observableArrayList();
    for (BestandView v : alleBestaende) {
        boolean passtName = suchtext.isEmpty()
        || v.getMaterialName().toLowerCase().contains(suchtext);
        boolean passtLager = lager == null
        || v.getLagerId() == lager.getId();
        if (passtName && passtLager) gefiltert.add(v);
    }
    bestandListe.setAll(gefiltert);
}

```

Listing 4: Filterung der Bestandsübersicht nach Name und Station

5.5 Mindestbestand-Warnung und Lagerbuchungen

Liegt der Bestand auf oder unter dem Mindestbestand, färbt eine RowFactory die Zeile rot:

```

tabBestand.setRowFactory(tv -> new TableRow<BestandView>() {
    @Override protected void updateItem(BestandView item,
        boolean empty) {
        super.updateItem(item, empty);
        if (empty || item == null) setStyle("");
        else if (item.isWarnung())
            setStyle("-fx-background-color: #FADBD8;");
        else setStyle("");
    }
}); // isWarnung(): bestand <= mindestbestand

```

Listing 5: Mindestbestand-Warnung über eine RowFactory

Bestandsänderungen erfolgen ausschließlich über Buchungen. Über die Schaltfläche „Warenentnahme“ wird ein AUSGANG gebucht, über „Umlagerung“ zwischen zwei Lagern ein AUSGANG im Quell- und ein EINGANG im Ziellager (Doppelbuchung). Auf der Registerkarte Bestandsbewegungen lassen sich Ein- und Ausgänge zusätzlich manuell anlegen, bearbeiten und löschen. Der Status einer Bestellung (offen, geliefert, storniert) dokumentiert nur den Bestellvorgang; den zugehörigen Wareneingang bucht der Nutzer anschließend als EINGANG.

6 Abnahme- und Testphase

6.1 Testfälle und Testergebnisse

Zur Qualitätssicherung wurden zehn Testfälle definiert, die alle Pflicht- und Zusatzanforderungen abdecken. Jeder Testfall wird manuell durchgeführt und das Ergebnis protokolliert (bestanden / fehlgeschlagen). Erwartet wird, dass alle Pflicht-Testfälle bestehen, da die Funktionen umgesetzt und die in Phase 6 gefundenen Fehler behoben wurden.

Tabelle 8: Definierte Testfälle (Auszug)

#	Testfall	Erwartetes Ergebnis	Prio
T1	Material anlegen	Erscheint in Tabelle, DB-Eintrag vorhanden	Hoch
T2	Material bearbeiten	Geänderte Werte in Tabelle und DB	Hoch
T3	Material löschen	Aus Tabelle und DB entfernt	Hoch

#	Testfall	Erwartetes Ergebnis	Prio
T4	Mindestbestand-Warnung	Zeile rot bei Bestand $\leq$ Mindestbestand	Hoch
T5	Suche nach Name	Tabelle filtert live beim Tippen	Mittel
T6	Filter nach Station	Nur Material der gewählten Station	Mittel
T7	AUSGANG buchen	Bestand sinkt, Buchung in Historie	Hoch
T8	EINGANG buchen	Bestand steigt, Buchung in Historie	Mittel
T10	Pflichtfeld leer	Fehlermeldung, kein DB-Insert	Hoch

Während der Umsetzung wurden mehrere Fehler gefunden und behoben: ein falsch benannter Primärschlüssel in der BestandsbewegungDAO, ein falscher Datenbankname bzw. Port in der DBConnection sowie ein nicht im Schema vorgesehener ENUM-Wert (TRANSFER). Alle Fehler wurden über die Fehlermeldungen lokalisiert und im Projekttagbuch mit Ursache und Lösung dokumentiert.

6.2 Soll-Ist-Vergleich

Tabelle 9: Soll-Ist-Vergleich der Anforderungen

Anforderung	Umsetzung	Status
MySQL $\geq$ 5 Tabellen	6 Tabellen umgesetzt	erfüllt
CRUD-Operationen	Vollständig für alle 6 Entitäten	erfüllt
MVC-Architektur	Model / View / Controller + DAO	erfüllt
Such-/Filterfunktion	Suche nach Name, Filter nach Station	erfüllt
Mindestbestand-Warnung	Rote Zeilenfärbung (RowFactory)	erfüllt
Buchungshistorie	Tabelle bestandsbewegungen	erfüllt

6.3 Abnahme und Übergabe

Die formale Abnahme erfolgt durch Frau Cramer am Ende von Block 2 (KW 26, bis 26.06.2026). Grundlage ist die vollständige Umsetzung der Pflichtanforderungen: lauffähige Anwendung, stabile Datenbankverbindung, korrekte Such- und Filterfunktion, sichtbare Mindestbestand-Warnung sowie eine vollständige Dokumentation. Übergeben werden der Quellcode (Maven-Projekt, Git), das Datenbankskript (Anhang B), das Benutzerhandbuch (Anhang C), das ER-Diagramm (Anhang A) und die Projektdokumentation als PDF (Moodle).

Bekannte Einschränkungen der aktuellen Version: keine Benutzerverwaltung mit Login, keine automatische Nachbestellung (das System warnt nur), keine Netzwerkfunktion und kein Export nach PDF oder Excel.

7 Dokumentation

7.1 Benutzerhandbuch

Für die Endnutzer wurde ein ausführliches Benutzerhandbuch erstellt. Es erklärt Schritt für Schritt den Programmstart, die Navigation und jede Funktion – vom Anlegen eines Materials über das Suchen und Filtern bis zur Buchungshistorie – und enthält einen FAQ-Teil zur Fehlerbehebung. Das vollständige Handbuch befindet sich in Anhang C.

7.2 Entwicklerdokumentation

Für die technische Nachvollziehbarkeit ist der Quellcode kommentiert und nach dem MVC- und DAO-Muster klar gegliedert (sprechende Klassen- und Methodennamen). Eine README beschreibt Aufbau und Start des Projekts, das Datenbankskript legt Schema und Testdaten an, und das Projekttagbuch hält alle Entscheidungen, Probleme und Lösungen fest. Damit kann die Anwendung später nachvollzogen und erweitert werden.

8 Fazit

8.1 Soll-Ist-Vergleich der Projektziele

Die Kernziele des Projekts wurden erreicht: Alle vier Pflichtenforderungen sind umgesetzt, die sechs Tabellen, das vollständige CRUD, die MVC-Architektur mit DAO-Schicht sowie Suche, Filter und Mindestbestand-Warnung funktionieren. Der Zeitplan hat sich als realistisch erwiesen; der eingeplante Puffer gab in der Test- und Dokumentationsphase Spielraum. Unterm Strich hat des gut geklappt.

8.2 Lessons Learned

Aus dem Unterricht (Dino-Park-Beispiel von Frau Cramer) waren das DAO-Muster, die Singleton-Verbindung und der Aufbau von Model-Klassen bekannt. Darauf aufbauend habe ich mir selbstständig die JavaFX-Themen erarbeitet: TableView mit ObservableList, eine gefilterte Bestandssicht, Dialoge, TabPane und die Zeilenfärbung über eine RowFactory. Auf SQL-Ebene war die Berechnung des Bestands über eine Aggregation aus den Bewegungen eine wichtige Erkenntnis. Besonders hilfreich war ein strukturiertes Vorgehen bei Fehlern: Meldung lesen, Ursache eingrenzen, Lösung dokumentieren – und das Projekttagbuch konsequent pflegen.

8.3 Ausblick

Für eine produktive Nutzung wären eine Benutzerverwaltung mit Login und Rollen, eine Backup-Funktion sowie ein Export nach PDF und Excel sinnvolle Erweiterungen. Außerdem würde ich Testfälle künftig früher und begleitend zur Entwicklung umsetzen und der Oberfläche von Beginn an ein einheitliches CSS-Stylesheet zugrunde legen. Diese Punkte wurden im aktuellen Zeitrahmen bewusst als optional eingestuft.

Abgabeerklärung

Ich erkläre hiermit, dass ich diese Projektdokumentation selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe.

Ort, Datum	Unterschrift (Sascha Schulz)
------------	------------------------------

Quellenverzeichnis

- [1] Oracle Corporation: Java SE 17 Documentation. Oracle, 2021. URL: <https://docs.oracle.com/en/java/javase/17/> (Abruf: 04.05.2026)
- [2] Oracle / OpenJFX: JavaFX 21 API Documentation. 2021. URL: <https://openjfx.io/javadoc/21/> (Abruf: 04.05.2026)
- [3] Oracle / MySQL: MySQL 8.0 Reference Manual. 2023. URL: <https://dev.mysql.com/doc/refman/8.0/en/> (Abruf: 04.05.2026)
- [4] SRH Berufliche Rehabilitation GmbH: DOIT-Leitfaden, Gruppe 2551. Unveröffentlichtes internes Dokument, Heidelberg, 2024.
- [5] Gamma, E. / Helm, R. / Johnson, R. / Vlissides, J.: Design Patterns. Addison-Wesley, Boston, 1994.
- [6] Oracle / OpenJFX: JavaFX 21 – TableView (API). URL: <https://openjfx.io/javadoc/21/javafx.controls/javafx/scene/control/TableView.html> (Abruf: 07.06.2026)
- [7] Oracle / OpenJFX: JavaFX 21 – ObservableList (API). URL: <https://openjfx.io/javadoc/21/javafx.base/javafx/collections/ObservableList.html> (Abruf: 07.06.2026)
- [8] Oracle / OpenJFX: JavaFX 21 – TabPane (API). URL: <https://openjfx.io/javadoc/21/javafx.controls/javafx/scene/control/TabPane.html> (Abruf: 07.06.2026)
- [9] Oracle / OpenJFX: JavaFX 21 – Dialog (API). URL: <https://openjfx.io/javadoc/21/javafx.controls/javafx/scene/control/Dialog.html> (Abruf: 07.06.2026)
- [10] Oracle: Introduction to FXML (JavaFX-Tutorial). URL: https://docs.oracle.com/javase/8/javafx/fxml-tutorial/get_started.htm (Abruf: 07.06.2026)
- [11] Apache Software Foundation: Maven – Getting Started Guide. URL: <https://maven.apache.org/guides/getting-started/> (Abruf: 07.06.2026)
- [12] Oracle: MySQL Connector/J Developer Guide. URL: <https://dev.mysql.com/doc/connector-j/en/> (Abruf: 07.06.2026)
- [13] Oracle: Java SE 8 API – java.sql.PreparedStatement. URL: <https://docs.oracle.com/javase/8/docs/api/java/sql/PreparedStatement.html> (Abruf: 07.06.2026)

Anhang

Anhang	Titel	Beschreibung
A	ER-Diagramm	Datenmodell der 6 Tabellen
B	SQL-Datenbankskript	CREATE-Statements aller Tabellen
C	Benutzerhandbuch	Vollständiges, detailliertes Handbuch
D	Wochenplan	Wochenplan_Sascha_Schulz.xlsx
E	Projektantrag	Projektantrag_Sascha_Schulz_v2.docx

Anhang A – ER-Diagramm

Das ER-Diagramm zeigt alle sechs Tabellen mit Entitäten, Attributen, Primär- und Fremdschlüsseln sowie den Kardinalitäten (1:N). Der Bestand wird nicht gespeichert, sondern aus bestandsbewegungen berechnet.

ER-Diagramm: Materialverwaltung Krankenhaus (6 Tabellen, 3NF)

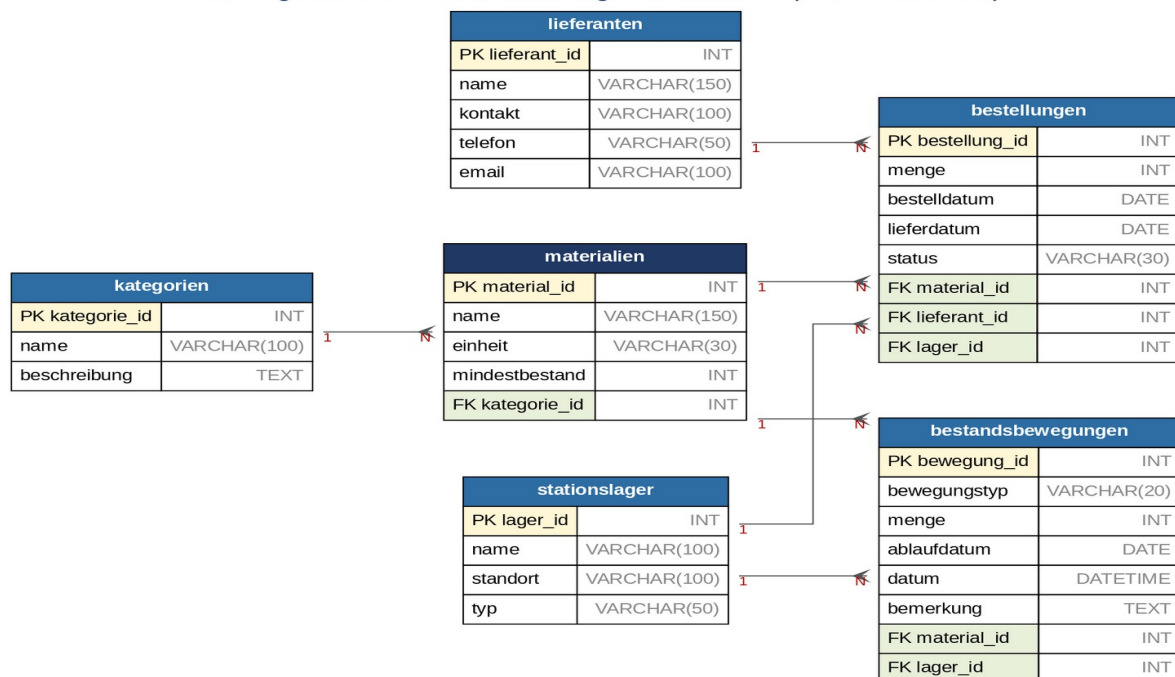


Abbildung 5: ER-Diagramm der Datenbank (Anhang A)

Anhang B – SQL-Datenbankskript

Datenbank krankenhaush_lager (utf8mb4). Nachfolgend die CREATE-Statements aller sechs Tabellen; das vollständige Skript inkl. Testdaten liegt in testdaten_krankenhaus_lager.sql bei.

```
CREATE DATABASE krankenhaush_lager
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE krankenhaush_lager;

CREATE TABLE kategorien (
  kategorie_id INT AUTO_INCREMENT PRIMARY KEY,
```

```

name VARCHAR(100) NOT NULL, beschreibung TEXT );

CREATE TABLE stationslager (
  lager_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL, standort VARCHAR(100), typ VARCHAR(50) );

CREATE TABLE lieferanten (
  lieferant_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(150) NOT NULL, kontakt VARCHAR(100),
  telefon VARCHAR(50), email VARCHAR(100) );

CREATE TABLE materialien (
  material_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(150) NOT NULL, einheit VARCHAR(30) NOT NULL,
  mindestbestand INT NOT NULL DEFAULT 0, kategorie_id INT NOT NULL,
  FOREIGN KEY (kategorie_id) REFERENCES kategorien(kategorie_id) );

CREATE TABLE bestellungen (
  bestellung_id INT AUTO_INCREMENT PRIMARY KEY,
  material_id INT NOT NULL, lieferant_id INT NOT NULL,
  lager_id INT NOT NULL, menge INT NOT NULL,
  bestelldatum DATE NOT NULL, lieferdatum DATE,
  status VARCHAR(50) NOT NULL DEFAULT 'offen',
  FOREIGN KEY (material_id) REFERENCES materialien(material_id),
  FOREIGN KEY (lieferant_id) REFERENCES lieferanten(lieferant_id),
  FOREIGN KEY (lager_id) REFERENCES stationslager(lager_id) );

CREATE TABLE bestandsbewegungen (
  bewegung_id INT AUTO_INCREMENT PRIMARY KEY,
  material_id INT NOT NULL, lager_id INT NOT NULL,
  bewegungstyp ENUM('EINGANG','AUSGANG') NOT NULL,
  menge INT NOT NULL, ablaufdatum DATE,
  datum DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP, bemerkung TEXT,
  FOREIGN KEY (material_id) REFERENCES materialien(material_id),
  FOREIGN KEY (lager_id) REFERENCES stationslager(lager_id) );

```

Listing 6: Datenbankschema – CREATE-Statements aller sechs Tabellen

Anhang C – Benutzerhandbuch

Materialverwaltung für medizinisches Verbrauchsmaterial – Krankenhaus-Edition (JavaFX + MySQL). Version 1.0. Zielgruppe: Pflege- und Stationspersonal sowie Verwaltung. Dieses Handbuch führt Schritt für Schritt durch alle Funktionen.

C.1 Systemvoraussetzungen

Komponente	Anforderung
Betriebssystem	Windows 10 oder 11 (64-Bit)
Java Runtime	Java 17 oder neuer (JRE/JDK)
Datenbankserver	MySQL 8.x – Dienst muss laufen
Datenbank	krankenhaus_lager – muss angelegt sein
Auflösung / RAM	min. 1280 × 768; min. 4 GB RAM

C.2 Programmstart

So starten Sie die Anwendung:

1. Prüfen Sie, ob der MySQL-Dienst läuft (Windows-Dienste > „MySQL“ > Status „Wird ausgeführt“).
2. Starten Sie das Programm per Doppelklick auf KrankenhausLager.jar oder über Main.java in der IDE.
3. Das Hauptfenster öffnet sich und zeigt automatisch die Registerkarte „Bestandsübersicht“.
4. Erscheint die Meldung „Datenbankverbindung fehlgeschlagen“, starten Sie den MySQL-Dienst und prüfen die Zugangsdaten in DBConnection.java.

Das Fenster gliedert sich in die Tab-Leiste (oben), die Such- und Filterleiste, die Tabelle (Mitte) und die Schaltflächen Hinzufügen, Bearbeiten, Löschen und Aktualisieren (unten).

Registerkarte	Funktion
Bestandsübersicht	Startansicht: Bestand je Material und Lager, Suche, Stationsfilter, Warnung, Warenentnahme, Umlagerung
Materialien	Stammdaten der Materialien verwalten (Kern)
Kategorien	Materialgruppen verwalten
Stationslager	Stationen und Lager verwalten
Lieferanten	Lieferantenstammdaten (optional)
Bestellungen	Nachbestellungen erfassen und verfolgen (optional)
Bestandsbewegungen	Ein- und Ausgänge manuell buchen und nachvollziehen

C.3 *Material anlegen*

5. Öffnen Sie die Registerkarte „Materialien“.
6. Klicken Sie unten auf „Hinzufügen“. Es öffnet sich ein Dialogfenster.
7. Feld „Name“: Materialbezeichnung eingeben, z. B. Einmalhandschuhe M.
8. Feld „Einheit“: Mengeneinheit eingeben, z. B. Stk, Pkg oder Fl.
9. Feld „Mindestbestand“: Zahl eingeben, ab welcher gewarnt wird, z. B. 100.
10. Feld „Kategorie-ID“: die Nummer der gewünschten Kategorie eingeben, z. B. 1 (die IDs sehen Sie auf der Registerkarte Kategorien).
11. Auf „Speichern“ klicken. Das Material erscheint sofort in der Tabelle.

Hinweis: Leere Pflichtfelder werden rot markiert; der Datensatz wird dann nicht gespeichert. Ein Material ist eine reine Stammdatei – der Bestand ergibt sich aus den Buchungen, nicht aus dem Material selbst.

C.4 *Material bearbeiten und löschen*

Bearbeiten: Zeile in der Tabelle anklicken › „Bearbeiten“ (oder Doppelklick) › Werte im Dialog ändern › „Speichern“. Die Tabelle aktualisiert sich automatisch.

Löschen: Zeile anklicken › „Löschen“ › Sicherheitsabfrage mit „Ja“ bestätigen. Achtung: Gelöschte Datensätze lassen sich nicht wiederherstellen. Lässt sich ein Material nicht löschen, bestehen noch abhängige Bestellungen oder Buchungen, die zuerst entfernt werden müssen.

C.5 *Suchen und Filtern*

12. Öffnen Sie die Registerkarte „Bestandsübersicht“.
13. Tippen Sie im Suchfeld einen Materialnamen (z. B. Handschuh) und drücken Sie Enter oder die Schaltfläche „Suchen“. Groß-/Kleinschreibung spielt keine Rolle.
14. Wählen Sie im Dropdown „Station“ eine Station, um nur deren Bestände zu sehen.
15. Such- und Stationsfilter wirken zusammen. „Zurücksetzen“ leert beide Filter, „Aktualisieren“ lädt die Daten neu.

C.6 *Mindestbestand-Warnung verstehen*

In der Bestandsübersicht wird die gesamte Zeile rot eingefärbt, sobald der Bestand auf oder unter dem Mindestbestand liegt. Das ist das Signal, dieses Material dringend nachzubestellen. Die Färbung passt sich automatisch an, sobald durch eine Buchung wieder genug Bestand vorhanden ist.

C.7 Kategorien, Stationslager und Lieferanten

Diese Stammdaten werden auf den gleichnamigen Registerkarten gepflegt – jeweils mit denselben Schaltflächen Hinzufügen, Bearbeiten und Löschen. Jedes Material muss einer Kategorie zugeordnet sein; eine Kategorie lässt sich nur löschen, wenn ihr kein Material mehr zugeordnet ist. Stationslager bilden Stationen und Lager ab (z. B. Station 1, OP, ICU, Zentrallager). Lieferanten (optional) speichern die Stammdaten der Bezugsfirmen.

C.8 Bestellungen erfassen

16. Registerkarte „Bestellungen“ öffnen › „Hinzufügen“.
17. Material und Lieferant im Dropdown wählen.
18. Menge und Bestelldatum eingeben. Der Status wird automatisch auf „offen“ gesetzt.
19. Auf Speichern klicken - die Bestellung erscheint in der Liste.
20. Sobald die Lieferung eintrifft: Bestellung auswählen und den Status auf geliefert setzen, dann speichern.

Hinweis: Das Setzen des Status verändert den Bestand nicht automatisch. Den Wareneingang buchen Sie anschliessend als EINGANG auf der Registerkarte Bestandsbewegungen (siehe C.9). Statuswerte: offen, geliefert, storniert.

C.9 Buchungen (Ein- und Ausgang)

Auf der Registerkarte Bestandsbewegungen werden alle Ein- und Ausgänge gebucht und protokolliert. In der Bestandsübersicht gibt es zusätzlich die Schnellzugriffe „Warenentnahme“ (AUSGANG) und „Umlagerung“. So buchen Sie eine Bewegung:

21. Registerkarte Bestandsbewegungen öffnen, dann Anlegen.
22. Material und Stationslager wählen.
23. Bewegungstyp EINGANG (Zugang) oder AUSGANG (Entnahme) auswählen.
24. Menge eingeben und optional eine Bemerkung (z. B. Entnahme OP).
25. Auf Speichern klicken. Datum und Uhrzeit werden automatisch gesetzt; der Bestand passt sich an.

Eine Umlagerung zwischen zwei Lagern erfassen Sie als AUSGANG im Quelllager und EINGANG im Ziellager. Nur durch konsequentes Buchen bleibt der Bestand lückenlos nachvollziehbar.

C.10 FAQ und Fehlerbehebung

Frage / Meldung	Loesung
Programm startet nicht	Java 17 installiert? (java -version). MySQL-Dienst aktiv? Zugangsdaten pruefen.
Datenbankverbindung fehlgeschlagen	MySQL-Dienst in den Windows-Diensten starten.
Access denied for user	Falsches Passwort - Zugangsdaten in DBConnection.java pruefen.
Zeile ist rot markiert	Bestand kleiner oder gleich Mindestbestand - Material nachbestellen.
Material laesst sich nicht loeschen	Erst abhaengige Bestellungen/Buchungen entfernen.
Geloeschte Daten zurueckholen?	Nicht moeglich - mit Bedacht loeschen.

Support: IT-Systemadministrator vor Ort oder Entwickler Sascha Schulz (Gruppe 2551).
Benutzerhandbuch Version 1.0 - SRH Berufliche Rehabilitation, WI-Ausbildung 2026.